27

Fine-Tuning

Teaching a ready-made model your specific task

Custom performance without training from scratch

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	Adapting a pre-trained model to your own data
Why	Custom results without huge data or cost
The idea	Keep the general knowledge, add your task
Modern way	LoRA — cheap, fast, light fine-tuning
Skill gap	One of your roadmap targets to master

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	**What is Fine-Tuning?**	Pre-training vs fine-tuning
2	**Why Fine-Tune?**	Custom results, little data
3	**How It Works**	Freezing and training layers
4	**Full vs Efficient Fine-Tuning**	The memory problem and LoRA
5	**LoRA — The Modern Way**	Cheap, fast fine-tuning
6	**The Fine-Tuning Steps**	Start to finish
7	**Fine-Tuning in Code**	A real Trainer example
8	**Fine-Tune or Just Prompt?**	When you actually need it
9	**Common Mistakes**	What to avoid
★	**Cheatsheet**	Quick reference

1. What is Fine-Tuning?

Fine-tuning means taking a model that was already trained on huge amounts of data, and training it a little bit more on YOUR specific data, so it gets good at YOUR specific task. You keep all the general knowledge it already has, and gently teach it your special skill.

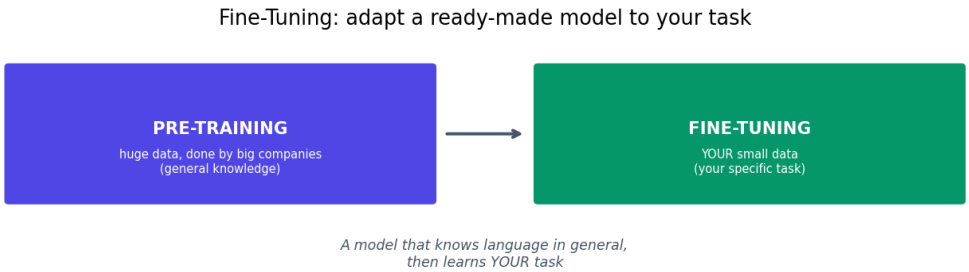

Fig 1 — Pre-training gives general knowledge (done by big companies). Fine-tuning adds your specific task.

Think of it like hiring someone who already speaks English fluently, then giving them a short training course on your company's specific products. You don't teach them English from zero — that took years. You just add the bit they need for your job. Fine-tuning does the same for a model: it already understands language; you teach it your particular task.

> ■ *Fine-tuning is one of the skill gaps on your roadmap. It's a key skill that separates people who just use models from people who can customise them — very valuable in job applications.*

2. Why Fine-Tune?

A general pre-trained model is good at general things. But your task might be specific: detecting defects in YOUR products, classifying YOUR customer messages, or writing in YOUR company's style. Fine-tuning adapts the model to do exactly that, far better than the general version.

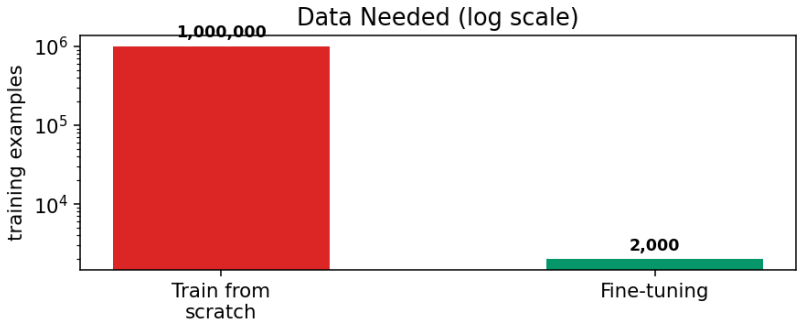

Fig 2 — Training from scratch needs millions of examples. Fine-tuning needs only a few thousand (or fewer).

The biggest benefit is how little data you need. Training a model from scratch needs millions of examples and huge cost. Fine-tuning can work with just a few thousand examples, sometimes even a few hundred, because the model already knows the basics. You're adding a specialty, not teaching from zero.

3. How It Works — Freezing Layers

Remember from the CNN topic that early layers learn general features and later layers learn task-specific ones. Fine-tuning uses this. You 'freeze' the early layers (lock them so they don't change) and only train the later layers on your data.

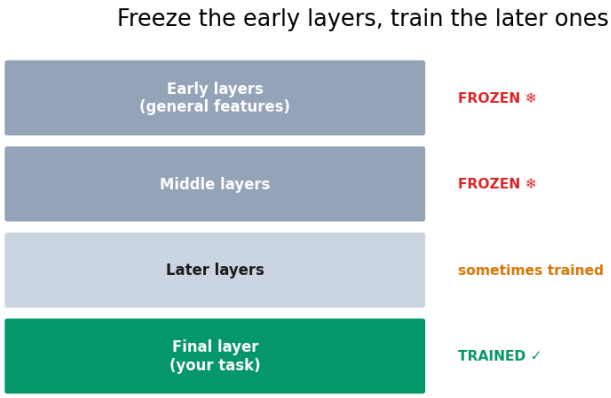

Fig 3 — Freeze the early layers (keep their general knowledge), train the later layers on your task.

Freezing the early layers protects the valuable general knowledge the model already has, and means there's far less to train — so it's faster and needs less data. The amount you freeze is a choice: freeze almost everything for tiny datasets, or unfreeze more layers if you have more data and want deeper adaptation.

> ■ *This is the same freeze-and-train idea from your image projects (ResNet, EfficientNet). Fine-tuning a language model works exactly the same way — you already know the core concept.*

4. Full vs Efficient Fine-Tuning

There's a catch with modern models: they're enormous. A model with billions of weights is too big to fully fine-tune on a normal computer — updating every weight needs huge memory. This led to a smarter approach.

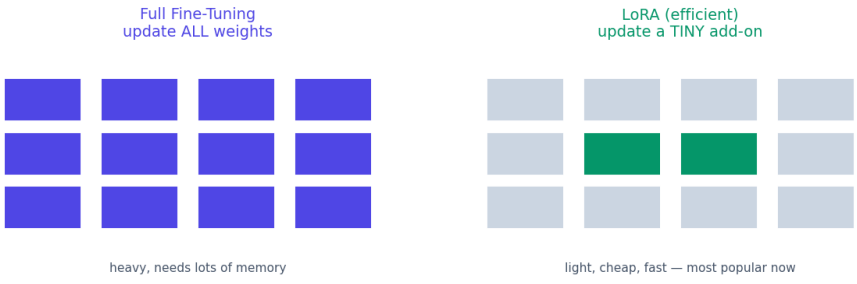

Fig 4 — Full fine-tuning updates all weights (heavy). LoRA adds a tiny trainable piece (light and cheap).

	Full Fine-Tuning	Efficient (LoRA)
What it updates	All the model's weights	A tiny added-on piece
Memory needed	Very high	Low — runs on small GPUs

Speed	Slow	Fast
Storage	A full model copy each time	A few MB per task
Popularity now	Rare for big models	The standard choice

5. LoRA — The Modern Way

LoRA (Low-Rank Adaptation) is the most popular efficient fine-tuning method. Instead of changing the model's billions of weights, it freezes them all and adds a small, trainable add-on. You only train that tiny add-on — often less than 1% of the model's size.

The result is almost as good as full fine-tuning, but it uses a fraction of the memory, trains much faster, and the saved result is only a few megabytes instead of gigabytes. This is what makes it possible to fine-tune large models on a single modest GPU, or even free cloud notebooks.

Benefit	Why it matters
Tiny memory use	Fine-tune big models on small GPUs
Fast training	Hours instead of days
Small output	A few MB, easy to store and share
Swappable	Keep many LoRA add-ons for one base model

■ *QLoRA is an even more memory-saving version that also shrinks the base model. Together, LoRA and QLoRA are why hobbyists can now fine-tune large models at home. Learning these is a strong, in-demand skill.*

6. The Fine-Tuning Steps

- **Step 1:** Pick a pre-trained model from Hugging Face that fits your task.
- **Step 2:** Prepare YOUR labelled data (text + labels, or your examples).
- **Step 3:** Tokenize the data with the model's matching tokenizer.
- **Step 4:** Set up training — freeze layers or apply LoRA, choose a small learning rate.
- **Step 5:** Train for a few epochs on your data, watching the validation score.
- **Step 6:** Evaluate, then save and use your custom model.

■ *Use a SMALL learning rate when fine-tuning (like 2e-5). The model is already good — you want gentle nudges, not big changes that wipe out its existing knowledge. Too high a rate causes 'catastrophic forgetting'.*

7. Fine-Tuning in Code

Hugging Face's Trainer makes fine-tuning manageable. Here's the shape of a text-classification fine-tune:

```
from transformers import (AutoTokenizer,
    AutoModelForSequenceClassification, Trainer, TrainingArguments)
from datasets import load_dataset

# 1. Model + tokenizer
name = 'distilbert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification.from_pretrained(
    name, num_labels=2)

# 2. Data, tokenized
data = load_dataset('imdb')
def tok(batch): return tokenizer(batch['text'],
    truncation=True, padding=True)
data = data.map(tok, batched=True)

# 3. Training settings (note the small learning rate)
args = TrainingArguments(
    output_dir='out', num_train_epochs=3,
    per_device_train_batch_size=16, learning_rate=2e-5)

# 4. Train
trainer = Trainer(model=model, args=args,
    train_dataset=data['train'], eval_dataset=data['test'])
trainer.train()

# 5. Save your custom model
trainer.save_model('my-finetuned-model')
```

■ *For LoRA, you add the PEFT library (pip install peft) and wrap the model in a few extra lines. The Trainer loop stays the same. PEFT is Hugging Face's toolkit for all the efficient fine-tuning methods.*

8. Fine-Tune or Just Prompt?

Important practical question: you don't always NEED to fine-tune. With big modern LLMs, sometimes just writing a good prompt (next topic) gets you what you want, with zero training. Here's how to decide.

Just prompt when...	Fine-tune when...
The base model already does it well	You need a consistent specific style/format
You have little or no labelled data	You have good labelled examples
You want quick, flexible results	You run the task at high volume (cheaper)
The task is general	The task is narrow and specialised

Practical order: try a good prompt first. If that's not consistent or accurate enough, then fine-tune. Fine-tuning gives more reliable, specialised behaviour and can be cheaper to run at scale, but prompting is faster to start and needs no data.

9. Common Mistakes

Mistake	Why it's a problem	Fix
Learning rate too high	Wipes out existing knowledge	Use a small rate (e.g. 2e-5)
Too few examples	Model doesn't learn the task	Get more, or just prompt instead
Full fine-tuning a huge model	Runs out of memory	Use LoRA / PEFT
Wrong tokenizer	Garbage inputs	Use the model's own tokenizer
Training too long	Overfits your small data	Few epochs, watch validation
Fine-tuning when prompting works	Wasted effort	Try a prompt first

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
Fine-tuning	train a pre-trained model on your data
Pre-training	the big general training (done for you)
Freezing	lock early layers, keep their knowledge
Full fine-tuning	update all weights (heavy)
LoRA	train a tiny add-on (light, popular)
QLoRA	even more memory-efficient LoRA
PEFT	HF library for efficient fine-tuning
Small learning rate	e.g. 2e-5 (gentle nudges)
Trainer	trainer = Trainer(model, args, data...)
Train	trainer.train()
Save	trainer.save_model('my-model')
Prompt vs fine-tune	try prompting first, fine-tune if needed

The one idea to remember: fine-tuning takes a model that already knows a lot and teaches it your specific task with a little of your own data. Freeze the general knowledge, train the task-specific part, and use LoRA to do it cheaply on modest hardware.

Next Topic: Sentence Transformers — models that turn whole sentences into meaningful number-vectors, capturing meaning so you can compare and search text by similarity. This powers your RAG chatbot.